

Programming Assignment # 2: From Two Binary Classifiers to Softmax Regression on the Iris Data Set

Assigned on September 24, 2026

Due on Canvas: October 7, 2026, 11:59PM

Total points: 100

1 Overview

Students will implement two ways of classifying the three Iris species with a linear model:

- (1) two binary logistic-regression classifiers chained together, and
- (2) a single multinomial (softmax) logistic-regression model.

Students will compare them and explain, in their own words, what they find.

Loading the data, splitting it, standardizing it, and scoring predictions – is given in `iris_utils.py`;

Students only write the models.

Learning objectives. After completing this homework students should be able to:

- implement the cross-entropy loss and gradient descent for binary logistic regression;
- combine two binary classifiers into a three-class prediction using a chain-rule argument;
- implement multinomial (softmax) logistic regression and compare it to the two-binary-classifier approach;
- read a confusion matrix and explain, using the data, where and why a classifier makes mistakes.

Logistics.

Item	Value
Submission	One PDF report and one code file per group uploaded to Canvas
Late policy	–10% per day, up to 3 days, –50% thereafter
Expected effort	5-7 hours

2 Ground rules

Implement the models yourself using only `numpy` for numerics and `matplotlib` for plotting. Use `iris_utils.py` exactly as given for loading, splitting, standardizing and scoring; you do not need to write or modify it;

Do not use `sklearn`, `scipy.optimize`, `statsmodels`, `torch`, `tensorflow` or any other library that would do the modeling work.

Everything you submit must run with only `numpy`, `matplotlib` and the provided `iris_utils.py`.

Get `iris.csv` and `iris_utils.py` from the assignment Canvas page. Both are plain text files (data: 150 rows of sepal/petal measurements and a species name; code: a handful of small numpy functions); Open and read them, that is part of understanding the pipeline.

Reproducibility. Use `seed = 0` whenever the instructions below call for “the” split. State any other seeds you use.

3 Background

Notation. n is the number of samples, $d = 4$ the number of (standardized) features.

`iris_utils.add_bias` prepends a constant 1 to every sample, giving $X_b \in \mathbb{R}^{n \times (d+1)}$;

The first weight multiplying that constant column is the bias, and it is never regularized.

Binary logistic regression. For labels $y_i \in \{0, 1\}$ and $p_i = \sigma(x_i^\top \theta)$ with $\sigma(z) = 1/(1 + e^{-z})$, minimize the L_2 -regularized mean cross-entropy

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right] + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2, \quad (1)$$

whose gradient (use this directly – no need to re-derive it) is

$$\nabla J(\theta) = \frac{1}{n} X_b^\top (p - y) + \lambda \tilde{\theta}, \quad \tilde{\theta} = (0, \theta_1, \dots, \theta_d), \quad (2)$$

with p the vector of all p_i . Batch gradient descent updates

$$\theta \leftarrow \theta - \eta \nabla J(\theta).$$

From three classes to two binary problems. So, we have two stages:

Stage 1 asks “is this setosa, or not?”

Stage 2, trained only on the non-setosa samples, asks “versicolor or virginica?”

With $q_1 = P(\text{setosa} \mid x)$ from stage 1 and $q_2 = P(\text{virginica} \mid x, \text{not setosa})$ from stage 2, the chain rule gives the three class probabilities:

$$P(\text{setosa}) = q_1, \quad P(\text{versicolor}) = (1 - q_1)(1 - q_2), \quad P(\text{virginica}) = (1 - q_1) q_2. \quad (3)$$

One cannot get this by training two classifiers **independently** and just picking the larger score – the second classifier’s output only means anything *given that the first one said “not setosa”*, which is exactly what the $(1 - q_1)$ factor encodes.

Softmax (multinomial) logistic regression. With $K = 3$ classes, weight matrix $W \in \mathbb{R}^{(d+1) \times 3}$ and one-hot labels $Y \in \{0, 1\}^{n \times 3}$, define $P = \text{softmax}(X_b W)$ row by row:

$$P_{ik} = \frac{e^{S_{ik}}}{\sum_{j=1}^3 e^{S_{ij}}}, \quad S = X_b W, \quad J(W) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^3 Y_{ik} \log P_{ik} + \frac{\lambda}{2} \sum_{j=1}^d \sum_{k=1}^3 W_{jk}^2, \quad (4)$$

with gradient (again, given directly – no derivation required for the core assignment)

$$\nabla_W J(W) = \frac{1}{n} X_b^\top (P - Y) + \lambda \widetilde{W}, \quad \widetilde{W} = W \text{ with its first (bias) row set to 0.} \quad (5)$$

Softmax solves the whole three-class problem in one shot; Part 3 asks you to compare it with the two-binary-classifier approach above.

4 Provided code (0 points)

`iris_utils.py` gives you, ready to use:

- `load_iris_csv(path)` – returns `X` (150×4) and `y` (150,) with species encoded 0/1/2;
- `stratified_split(X, y, test_fraction, seed)` – a seeded, class-balanced train/test split;
- `standardize(Xtr, Xte)` – scales both sets using statistics computed on the training set only (this avoids a subtle bug called *data leakage*: computing the scaling from the test set would let information about the test set influence training);
- `add_bias(X)`, `one_hot(y, K)` – small reshaping helpers;
- `accuracy(y_true, y_pred)`, `confusion_matrix(y_true, y_pred)` (rows = true class, columns = predicted class), and `multiclass_log_loss(P, y)` for scoring predictions.

Run this once at the top of your solution and reuse `Xtr`, `Xte`, `ytr`, `yte` throughout:

```
from iris_utils import *
X, y = load_iris_csv("iris.csv")
Xtr_raw, Xte_raw, ytr, yte = stratified_split(X, y, test_fraction=0.3, seed=0)
Xtr, Xte, mu, sd = standardize(Xtr_raw, Xte_raw)
```

This gives 105 training and 45 test samples, 35/15 per class.

5 Part 1 – Binary logistic regression (30 points)

1.1 (18 pts) Implement:

- a numerically stable `sigmoid(z)` (no overflow warning for $z = \pm 1000$: clip z to, say, $[-500, 500]$ before calling `np.exp`);

- `binary_loss(theta, Xb, y, lam)` and `binary_grad(theta, Xb, y, lam)` implementing Equations (1) and (2) (clip probabilities to $[10^{-12}, 1 - 10^{-12}]$ before taking a logarithm, so you never compute $\log 0$);
- a class `BinaryLogisticRegression` with `fit(X, y)`, `predict_proba(X)` and `predict(X)` (threshold 0.5), trained by batch gradient descent. `fit` should record the loss at every iteration in `self.loss_history` so you can plot it.

Use $\eta = 0.5$, $\lambda = 0.01$ and 2000 iterations unless told otherwise.

1.2 (12 pts) Train on the binary problem “setosa (label 1) vs. the rest (label 0)” with $\lambda = 0.1$. Report train and test accuracy, and plot the loss against the iteration number – it should decrease at every step. If it does not, something is wrong with your gradient or learning rate; fix it before moving on, since Parts 2 and 3 reuse this class.

6 Part 2 – Two binary classifiers vs. softmax (45 points)

2.1 (18 pts) Implement `TwoStageClassifier`, reusing your `BinaryLogisticRegression` from Part 1: `fit(X, y)` trains stage 1 (setosa vs. rest) on all training data, then trains stage 2 (versicolor vs. virginica) using only the non-setosa training rows. `predict_proba(X)` should return an $n \times 3$ array of probabilities computed with Equation (3) (check that each row sums to 1), and `predict(X)` should return the class with the highest probability.

2.2 (18 pts) Implement `softmax(S)` (subtract each row’s maximum before exponentiating, so it is stable for large inputs), `softmax_loss`, `softmax_grad` (Equations (4) and (5)), and a class `SoftmaxRegression` with `fit`, `predict_proba` and `predict`, trained the same way as Part 1’s binary model but with a weight *matrix* W instead of a weight vector. Use $\lambda = 0.01$, $\eta = 0.5$ and 3000 iterations.

2.3 (9 pts) Train both models on (X_{tr}, y_{tr}) and evaluate on (X_{te}, y_{te}) . Report, for each model: train accuracy, test accuracy, and the test confusion matrix.

7 Part 3 – Short-answer interpretation (25 points)

Answer each question in a few sentences, using your own numbers and the scatter plot of petal length against petal width for the training set, colored by species.

3.1 (8 pts) Look at the two confusion matrices from Task 2.3:

- Which pairs of classes get confused, if any?
- Is setosa ever involved in a mistake?
- Relate your answer to the scatter plot: does it look like the confused classes overlap in feature space?

3.2 (9 pts) In your own words, explain:

- Why `TwoStageClassifier` cannot just report “ $\max(q_1, q_2)$ ’s class” – why is the chain-rule combination in Equation (3) necessary?
- What role does the factor $(1 - q_1)$ play?

3.3 (8 pts) Based on your accuracy and confusion-matrix results, would you recommend the two-stage approach or softmax for this problem? There is no single right answer – justify your choice using your own numbers.

8 What to submit

One short PDF report (at most 4 pages, excluding code) and one code file.

Report. A brief introduction (one or two sentences), then one section per Part (1–3), answering each numbered task in order, with the task number as a subheading. Every figure needs axis labels and a one-line caption. End with the seeds you used (just `seed=0` unless you changed something).

Code. A single Python file that imports `iris_utils` and defines, at least, `sigmoid`, `binary_loss`, `binary_grad`, `BinaryLogisticRegression`, `TwoStageClassifier`, `softmax`, `softmax_loss`, `softmax_grad`, `SoftmaxRegression`. It should run top to bottom and print/plot everything your report refers to.

9 Grading rubric (100 points)

Task	Pts	Full credit requires	Typical partial credit
1.1 Binary implementation	18	Stable sigmoid (no overflow for $z = \pm 1000$), correct loss and gradient matching Eqs. 1–2, vectorized (no loop over samples), clipped log, <code>fit</code> records loss history, <code>predict</code> thresholds at 0.5	–4 for each of: overflow warnings, sample loops, bias regularized, no loss history
1.2 Setosa vs rest	12	Train and test accuracy reported (expect 1.000/1.000), monotonically decreasing loss curve with axis labels	6 if accuracy correct but curve missing or not monotone with no explanation
2.1 Two-stage	18	Stage 2 trained only on non-setosa rows; Eq. 3 implemented correctly (probabilities sum to 1); <code>predict</code> uses <code>argmax</code>	–6 if stage 2 trained on all data; –6 if probabilities do not sum to 1
2.2 Softmax	18	Stable softmax (row-max subtraction, no NaN for $[-1000, 0, 1000]$), correct loss/gradient matching Eqs. 4–5, vectorized over samples and classes, bias row unpenalized	–4 each: unstable softmax, sample/class loops, regularized bias
2.3 Comparison table	9	Train/test accuracy and confusion matrix for both models on the seed-0 split	–3 per missing quantity
3.1 Confusion analysis	8	Correctly identifies versicolor/virginica as the only confused pair; connects it to feature overlap in the scatter plot	4 for a correct but generic answer with no reference to the plot or numbers
3.2 Chain rule	9	Explains that q_2 is only meaningful conditional on “not setosa,” and that $(1 - q_1)$ carries that conditioning into an unconditional probability; notes that $\max(q_1, q_2)$ compares numbers that answer different questions	4–5 for a vague “it needs both classifiers” answer without the conditional-probability reasoning
3.3 Recommendation	8	A clear choice justified with the student’s own accuracy/confusion-matrix numbers from 2.3	4 for a choice with no supporting numbers
Code quality 0 (folded into task grading above)	–	–	–

Common mistakes. Regularizing the bias, training stage 2 on all samples instead of only the non-setosa ones, an unstable `softmax/sigmoid`

10 APPENDIX

11 Evaluation metrics from scratch

Every metric can be computed directly from integer counts. Use a 3×3 confusion matrix C whose rows are true classes and whose columns are predicted classes. State this orientation beside every displayed matrix.

$$\begin{aligned}\text{accuracy} &= \frac{\sum_k C_{kk}}{\sum_r \sum_c C_{rc}}, \\ \text{precision}_k &= \frac{C_{kk}}{\sum_r C_{rk}}, & \text{recall}_k &= \frac{C_{kk}}{\sum_c C_{kc}}, \\ \text{macro precision} &= \frac{1}{3} \sum_k \text{precision}_k, & \text{macro recall} &= \frac{1}{3} \sum_k \text{recall}_k.\end{aligned}$$

If a precision or recall denominator is zero, define that metric as zero and print a warning. With the required stratified test set, every true class has eight observations, but a model may predict no examples of a class.